

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1. ПОСТАНОВКА ЗАДАЧИ	6
2. КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....	9
3. РАЗРАБОТКА МОДЕЛИ ИССЛЕДОВАНИЯ, ОСНОВНЫЕ СООТНОШЕНИЯ.....	13
3.1. Структура базы данных	13
4. РАЗРАБОТКА СТРУКТУРЫ И ОПРЕДЕЛЕНИЕ ФУНКЦИЙ СИСТЕМЫ.....	16
4.1. Общая архитектура системы	16
4.2. Основные функции системы	17
4.3. Структура программных классов	17
4.3.1. Классы Attack и Probe.....	18
4.3.2. Сервисный класс AttackSimulationService	20
4.3.3. Контроллер AttacksController.....	22
5. СТРУКТУРА ИНТЕРФЕЙСА	25
6. СЦЕНАРИИ ПРОВЕДЕНИЯ ЭКСПЕРИМЕНТОВ	31
7. РЕЗУЛЬТАТЫ ИССЛЕДОВАНИЯ	33
ЗАКЛЮЧЕНИЕ	42
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	43
ПРИЛОЖЕНИЕ А.....	44

ВВЕДЕНИЕ

В рамках курсовой работы была поставлена задача разработать программную систему для моделирования и анализа методов обнаружения уязвимостей в интерфейсах информационно-измерительных сетей. Исследование направлено на изучение влияния различных параметров атак и стратегий тестирования на эффективность обнаружения уязвимостей.

Для достижения этой цели была разработана математическая модель, описывающая атакующие воздействия, включая линейные и экспоненциальные типы атак, а также процессы тестирования уязвимостей, такие как одиночное и коллаборационное тестирование. Реализованы алгоритмы для расчета вероятностей обнаружения уязвимостей, классификации уровней угроз и оптимизации параметров тестирования.

Центральным элементом работы является программный комплекс, позволяющий проводить вычислительные эксперименты, анализировать эффективность различных стратегий тестирования и визуализировать полученные результаты. Комплекс обеспечивает моделирование различных сценариев атак и тестирования, предоставляя инструменты для анализа и интерпретации результатов.

Объектом исследования являются процессы обнаружения уязвимостей в интерфейсах информационно-измерительных сетей, а предметом исследования - вероятностные методы обнаружения уязвимостей в условиях противодействия тестированию.

Работа включает введение, пять глав, заключение и список литературы. В ней рассматриваются теоретические основы, постановка задачи, разработка математических моделей, реализация программного комплекса и анализ результатов экспериментов.

1. ПОСТАНОВКА ЗАДАЧИ

Требуется исследовать влияние параметров интенсивности и продолжительности на:

- осуществление атаки;
 - противодействие тестированию.
1. Сформировать таблицу содержащей оценки классификации ситуаций обнаружения отказов при различных типах атак и их параметров.
 2. Построить графики изменения вероятностей отказа для заданных типов атак при $p(a_i) = 1.0$.
 3. Построить графики вероятности $P(p(t))$ обнаружения уязвимостей при тестировании отдельным устройством.
 4. Построить графики вероятности $P(p(t, k))$ обнаружения уязвимостей при коллаборационной стратегии для заданных значений параметров $p(t), n, k$.
 5. В соответствии с вариантами индивидуальных заданий провести модельные эксперименты с целью исследования изменения вероятности $P(p(t, k))$ обнаружения уязвимостей при коллаборационной стратегии в условиях противодействия тестированию. Получить оценки в зависимости от влияния следующих факторов:
 - тип атаки – линейная, экспоненциальная;
 - вероятности $p(a_i)$ появления i -ого типа атаки A_i в момент времени t ,
 - вероятности $P(p(t))$ однократного обнаружения уязвимости: $p(t) < 0.5$ и $p(t) > 0.5$;
 - различная мощность коллабораций и число тестируемых (n, k).

Считается, что для каждого типа атаки (линейная, экспоненциальная) рассматривается: Лин_Н, Лин_С, Лин_В и Exp_H , Exp_C , Exp_B . Соответственно вероятность появления для каждого типа атаки $p(a_i) = 0.33$.

Разработать жадный алгоритм формирования минимального покрытия на основе сгенерированной матрицы смежности $M(X, Y)$ размерности 15×15 и числом связей $|e(i, j)|$ между узлом X_i с узлами Y_j из диапазона $[1; 6]$.

В таблице 1.1 приведены варианты индивидуального задания исследования влияния параметров типов атак на вероятность отказа. Номера вариантов выбирались по следующим правилам, где I – порядковый номер в списке группы:

- $N = (I \bmod 11) + 1$,
- $N = (I \bmod 11) + 1$,
- $N = (I \bmod 11) + 1$

Таблица 1.1 – Типы и параметры атак

№ варианта	Функция			
	$p^A(t) = at + b$		$p^A(t) = 1 - a \times \exp(-bt)$	
	a	b	a	b
1	0.03	0.01	0.9	0.04
5	0.05	0.03	0.5	0.06
8	0.035	0.025	0.6	0.075

В таблице 1.2 приведены варианты индивидуального задания исследования вероятности обнаружения уязвимостей при коллаборационной стратегии. Номера вариантов выбираются по правилу: $N = (I \bmod 11) + 1$.

Таблица 1.2 – Вероятности однократного обнаружения уязвимости

№ варианта	Функция	
	$p(t) > 0.5$	$p(t) < 0.5$
5	0.9	0.41

2. КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Современные информационно-измерительные сети (ИИС) представляют собой сложные распределенные системы, включающие датчики, контроллеры и коммуникационные интерфейсы. В условиях активного внедрения технологий 5G, IoT и SDN (программно-определяемых сетей) возрастает количество потенциальных уязвимостей, связанных с:

- несанкционированным доступом к сетевым компонентам;
- изменением конфигураций измерительных устройств;
- внедрением вредоносного кода в управляющие системы;
- созданием преднамеренных помех в каналах связи.

В сетях с интеллектуальными беспилотными транспортными средствами (БТС) рассматриваются ситуации, когда злоумышленник стремится нарушить нормальное поведение узлов или системы в целом.

Введем следующие обозначения:

- A — осуществление атаки;
- R — атака обнаружена;
- U — осуществляется противодействие тестированию;
- W — противодействие обнаружено.

Тогда вероятности событий:

- $Pr(A, R)$ — атака есть и она обнаружена;
- $Qr(A, \bar{R})$ — атака есть, но не обнаружена;
- $Sr(\bar{A}, R)$ — атаки нет, но зафиксировано ложное срабатывание;
- $Tr(\bar{A}, \bar{R})$ — атаки нет и она не обнаружена.

Аналогично для противодействия тестированию:

- $Pu(U, W)$ — противодействие есть и обнаружено;
- $Qu(U, \bar{W})$ — противодействие есть, но не обнаружено;

- $Su(\bar{U}, W)$ — противодействия нет, но ложное срабатывание;
- $Tu(\bar{U}, \bar{W})$ — противодействия нет и оно не обнаружено.

Пусть вероятность обнаружения атаки за одно тестирование в момент времени t обозначается как $p(t)$. Если тестирование проводится k раз, вероятность обнаружить уязвимость именно на k -ом тесте выражается через геометрическое распределение:

$$P(p(t)) = p(t) \times (1 - p(t))^{k-1}, \quad (2.1)$$

В случае группового (коллораборационного) тестирования, когда из n устройств k участвуют в взаимной проверке, вероятность обнаружения уязвимости при биномиальном распределении формулируется как:

$$P(p(t), k, n) = \binom{n}{k} \times p(t)^k \times (1 - p(t))^{n-k}, \quad (2.2)$$

При этом коллораборационная стратегия предполагает, что каждый узел тестирует остальные, а решение принимается большинством (например, при числе успешных тестов $e \geq \frac{k}{2}$).

Для учета противодействия со стороны злоумышленника введем вероятность того, что атака типа Ai препятствует тестированию: $p^{Ai}(t)$. Тогда вероятность отсутствия противодействия — $1 - p^{Ai}(t)$.

С учетом противодействия выражения вероятности обнаружения уязвимости трансформируются:

$$P(p^A(t), p(t)) = (1 - p^A(t)) \times p(t) \times (1 - p(t))^{k-1} \quad (2.3)$$

$$P(p^A(t), p(t, k)) = (1 - p^A(t)) \times (C_n^k \times p^k(t) \times (1 - p(t))^{n-k}), \quad (2.4)$$

где C_n^k – биномиальный коэффициент, отражающий количество уникальных комбинаций k тестируемых устройств из n .

Если введем распределение типов атак и вероятность каждой атаки $p_{Ai}(t)$, то должно выполняться:

$$\sum_{i=1}^{N_a} p(Ai) = 1 \quad (2.5)$$

Условная вероятность отказа тестирования при атаке Ai – $p(\text{отк}|Ai)$.
Полная вероятность отказа тестирования при действии всех атак:

$$p^A(\text{отк}) = \sum_{i=1}^n p(Ai) \times p(\text{отк}|Ai) \quad (2.6)$$

Тогда модифицированные выражения вероятности обнаружения уязвимости в условиях активного противодействия приобретают вид:

$$P(p^A(t), p(t)) = (1 - \sum_{i=1}^n p(Ai) \times p(\text{отк}|Ai)) \times p(t) \times (1 - p(t))^{k-1} \quad (2.7)$$

$$P(p^A(t), p(t, k)) = (1 - \sum_{i=1}^n p(Ai) \times p(\text{отк}|Ai)) \times (C_n^k \times p^k(t) \times (1 - p(t))^{n-k}) \quad (2.8)$$

Значение вероятности действия атаки $p^A(t)$ оценивается интегрально с использованием шкалы, содержащей три интервала $[0; Z1; Z2; 1]$:

$$\begin{aligned}
0 \leq p^A(t) < Z1 &- \text{ низкий,} \\
Z1 \leq p^A(t) < Z2 &- \text{ средний,} \\
Z2 \leq p^A(t) < 1 &- \text{ высокий}
\end{aligned}
\tag{2.9}$$

Для оценки результатов моделирования установлены границы интервалов: $Z1 = 0.4, Z2 = 0.7$.

Для моделирования атак рассматриваются следующие зависимости вероятности атаки от времени:

1. Линейная модель

$$p^A(t) = at + b, \tag{2.10}$$

где a, b – параметры закона распределения, характеризующие интенсивность атаки.

2. Экспоненциальная модель:

$$p^A(t) = 1 - a \times \exp(-bt) \tag{2.11}$$

3. РАЗРАБОТКА МОДЕЛИ ИССЛЕДОВАНИЯ, ОСНОВНЫЕ СООТНОШЕНИЯ.

В рамках данной курсовой работы реализована программная модель оценки вероятности обнаружения атак в интеллектуальных сетях, включающая как симуляцию атакующих воздействий, так и расчет эффективности обнаружения с учетом различных стратегий тестирования. Архитектурно реализация выполнена на платформе Ruby on Rails, где логика моделирования атак инкапсулирована в модели Attack и Probe, а расчёты и агрегация данных — в сервисном объекте AttackSimulationService.

Модель направлена на решение задач моделирования атак с различной интенсивностью и вероятностью воздействия на систему, а также анализ устойчивости системы к этим воздействиям. Ключевым элементом является использование формализованных уравнений вероятностного характера, взятых из методических указаний к исследованию.

3.1. Структура базы данных

Для хранения и обработки данных о атаках и результатах тестирования используется СУБД PostgreSQL, выбранная благодаря:

- надежной системе типов данных;
- поддержке сложных запросов;
- возможностям масштабирования;
- интеграции с Ruby on Rails через ActiveRecord.

В таблице ниже представлено описание используемых моделей и их полей:

Таблица 3.1. – Структура базы данных

Модель	Поле	Тип	Назначение
User	first_name	string	Имя пользователя
	last_name	string	Фамилия пользователя
	email	string	Уникальный идентификатор для аутентификации
Attack	attack_type	string	Тип атаки: linear или exponential
	parameter_a	decimal(10,5)	Параметр а в уравнении вероятности атаки
	parameter_b	decimal(10,5)	Параметр b в уравнении вероятности атаки
Probe	attack_id	references	Связь с конкретной атакой
	time_step	integer	Временной шаг, t
	probability	float	Базовая вероятность обнаружения
	total_devices	integer	Общее количество устройств (n)
	tested_devices	integer	Количество тестируемых устройств (k)

На рисунке ниже приведена ER-диграмма используемой базы данных для разработанной системы.

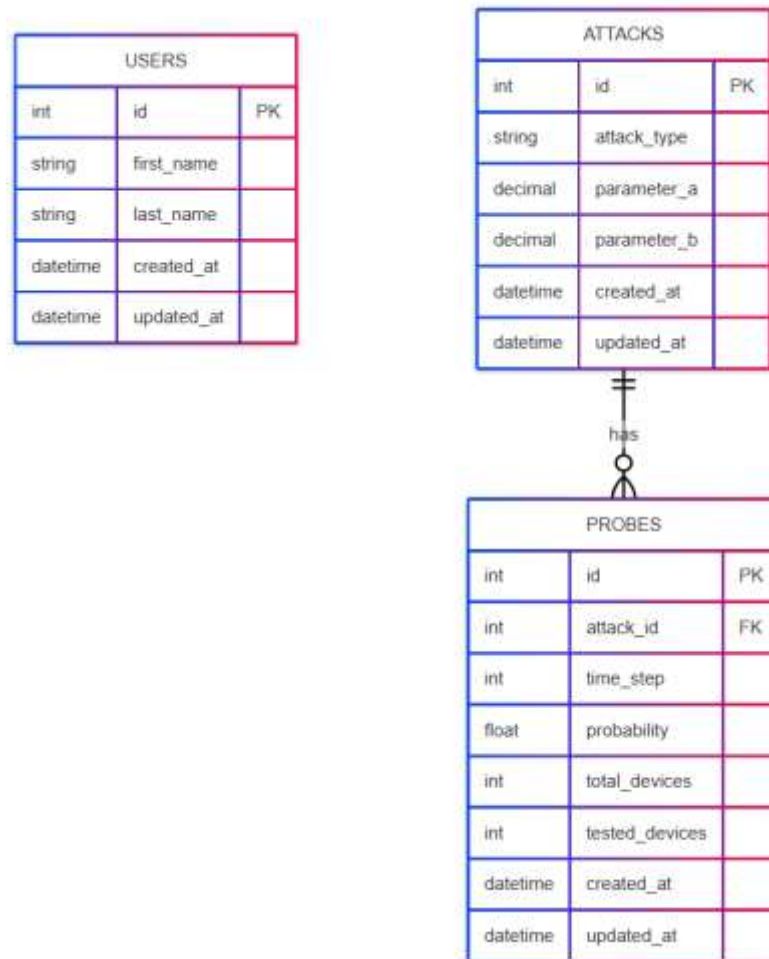


Рисунок 3.1 – ER-диаграмма

4. РАЗРАБОТКА СТРУКТУРЫ И ОПРЕДЕЛЕНИЕ ФУНКЦИЙ СИСТЕМЫ

4.1. Общая архитектура системы

Разрабатываемая программная система представляет собой веб-приложение, реализующее моделирование атак в интеллектуальных сетях и оценку вероятности их обнаружения в зависимости от ряда параметров. Архитектура системы основана на MVC-подходе, реализованном в фреймворке Ruby on Rails, что обеспечивает строгую организацию компонентов и разделение ответственности между уровнями.

Основные структурные компоненты включают (рисунок 4.1):

- Модели (Model) — описывают предметную область, хранят параметры атак и обеспечивают математические расчёты.
- Контроллеры (Controller) — обрабатывают действия пользователя и управляют потоком данных между моделью и представлением.
- Представления (View) — реализуют визуальное отображение результатов моделирования и обеспечивают пользовательское взаимодействие.
- Сервисы (Service Objects) — выделены для инкапсуляции сложной бизнес-логики симуляции.

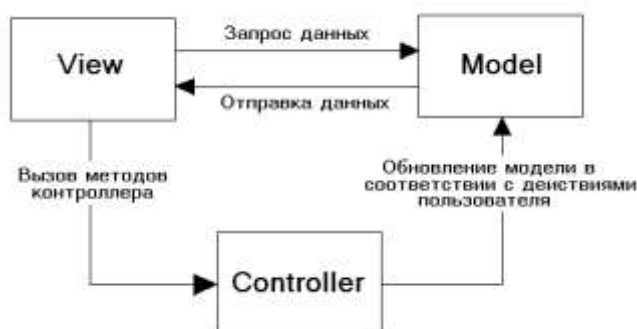


Рисунок 4.1 – Модель проектирования MVC

Также реализован веб-интерфейс, позволяющий задавать параметры моделирования, визуализировать графики вероятностей и анализировать результаты.

4.2. Основные функции системы

Описание функциональных возможностей разработанной системы приведено в таблице ниже.

Таблица 4.2 – Описание функциональных возможностей системы

Функция	Описание
Создание модели атаки	Задание параметров a и b , типа атаки (линейная или экспоненциальная).
Автоматическое моделирование	Проведение симуляции по заданным диапазонам параметров: временные шаги, число устройств.
Расчёт вероятности атаки	Вычисление вероятности в зависимости от времени и типа модели (линейная, экспоненциальная).
Оценка вероятности обнаружения	Расчёт по двум стратегиям: одиночное и коллаборативное тестирование.
Классификация уровня атак	Определение уровня угрозы на основе рассчитанной вероятности (низкий, средний, высокий).
Визуализация результатов	Построение таблиц, цветовая градация и графики результатов по временным шагам.
Управление пользователями	Регистрация, аутентификация, сохранение сессий и истории симуляций.

4.3. Структура программных классов

В основу архитектуры программной системы положена предметно-ориентированная модель, реализованная в виде совокупности классов, описывающих ключевые сущности исследуемой задачи. Каждый класс инкапсулирует свойства и поведение, присущие соответствующему

компоненту модели угроз и процесса обнаружения атак. Основное внимание уделено моделям *Attack* и *Probe*, а также сервисным объектам, отвечающим за вычисления и симуляции. Ниже представлено подробное описание ключевых классов.

4.3.1. Классы *Attack* и *Probe*

Класс *Attack* представляет собой модель атаки, параметризуемую коэффициентами a и b , а также типом функции роста (*linear* или *exponential*). Он отвечает за моделирование вероятности атаки во времени и за генерацию набора значений для дальнейшего анализа.

Атрибуты класса включают:

- *attack_type* — тип модели атаки (линейная или экспоненциальная), представленный в виде символьного значения;
- *parameter_a* — коэффициент наклона прямой или начальное значение в экспоненциальной функции;
- *parameter_b* — свободный коэффициент (для линейной модели) либо показатель экспоненты.

Методы класса:

- *linear_probability(time_step)* — реализует вероятностную модель линейного роста, заданную формулой 2.10;
- *exponential_probability(time_step)* — описывает экспоненциальную модель, при которой вероятность атаки асимптотически приближается к единице по формуле 2.11;
- *probability_at(time_step)* — универсальный интерфейс, делегирующий вызов соответствующей функции в зависимости от заданного типа модели атаки. Позволяет абстрагироваться от

конкретной реализации и использовать единый API при анализе угроз.

- *attack_level(time_step)* — интерпретирует рассчитанную вероятность атаки в категориальный уровень угрозы, выделяя три градации согласно формуле 2.9.

Класс *Attack* является ядром вероятностной модели угроз, генерируя значения, используемые в дальнейших расчетах. Он взаимодействует с классом *Probe*, обеспечивая исходные данные для оценки эффективности стратегий обнаружения. Благодаря параметризации и поддержке различных типов роста, *Attack* позволяет проводить сравнительный анализ различных сценариев атак, варьируя скорость и интенсивность развития угроз.

Класс *Probe* моделирует попытку обнаружения атаки в интеллектуальной сети при заданных параметрах. Он связывается с конкретной атакой через внешний ключ *attack_id* и хранит параметры тестирования, включая:

- *time_step* — момент времени, на который проводится проверка (обозначается t);
- *tested_devices* — количество устройств, подвергнутых тестированию (обозначается k);
- *total_devices* — общее число устройств в сети (обозначается n);
- *probability* — вероятность атаки в данный момент времени, полученная из соответствующей модели *Attack*.

Ключевыми методами класса являются:

- *single_test_probability(p)* — рассчитывает вероятность обнаружения атаки при одиночном тестировании, согласно формуле 2.3, где p — вероятность успешного обнаружения одной атакованной ноды;

- *collaborative_test_probability(p)* — реализует альтернативную стратегию согласно формуле 2.4, учитывающую скоординированное тестирование с учётом полной популяции n и возможности перекрытия проверок;
- *level* — возвращает уровень опасности, аналогично классификации в *Attack*, но на основе расчётов вероятности обнаружения.

Класс *Probe* обеспечивает эмпирическую реализацию моделей тестирования, выступая связующим звеном между теоретической моделью атаки и практической стратегией анализа угроз.

На рисунке ниже приведена UML-диаграмма классов.

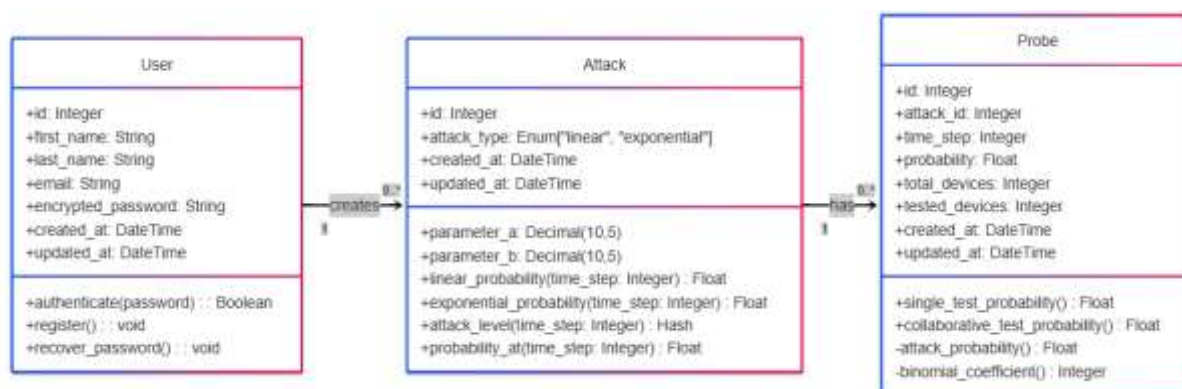


Рисунок 4.1 – UML-диаграмма

4.3.2. Сервисный класс *AttackSimulationService*

Сервисный объект *AttackSimulationService* инкапсулирует бизнес-логику, связанную с автоматизированным проведением симуляций вероятностных атак и моделированием процессов их обнаружения. Его основное назначение — централизованно обрабатывать генерацию набора объектов *Probe*, охватывая все релевантные комбинации параметров

тестирования и конфигурации сети.: времени t , количеству тестируемых устройств k , и размеру сети n .

Вынесение этой логики в отдельный слой служебных классов (service layer) позволяет разгрузить модели и контроллеры, обеспечивая высокую степень повторного использования кода, улучшение читаемости и соблюдение принципов чистой архитектуры (Separation of Concerns).

Сервис работает на основе:

- объекта модели *Attack*, который определяет сценарий атаки;
- диапазона дискретных временных шагов *time_steps*;
- диапазонов размеров сети *total_devices* и числа проверяемых узлов *tested_devices*;
- одной или нескольких фиксированных вероятностей обнаружения атакованной ноды (*probability_gt*, *probability_lt*), используемых для оценки стратегий обнаружения.

Каждая симуляция моделирует, как вероятность атаки (рассчитанная моделью *Attack*) соотносится с вероятностью её обнаружения (моделируемой через *Probe*) при различных конфигурациях сети.

Основные методы

- 1 *self.run(attack, time_steps, total_devices, tested_devices, probability_gt, probability_lt)* – главная точка входа для запуска симуляции. Возвращает результирующую структуру, содержащую массивы объектов *Result*, соответствующие каждому из сценариев обнаружения: при вероятности больше (*gt*) и меньше (*lt*) заданного порога. При отсутствии валидного объекта *attack* возвращается пустая структура.
- 2 *self.simulate(attack, time_steps, total_devices, tested_devices, probability)* – метод, выполняющий перебор всех возможных комбинаций параметров:

- $t \in [1, time_step]$ — момент времени;
- $k \in [1, tested_devices]$ — количество тестируемых устройств;
- $n \in [1, total_devices]$ — общее число устройств в сети, при условии $k < n$.

Для каждой комбинации создаётся объект *Probe*, в который передаются:

- 1 *attack* — ссылка на модель атаки;
- 2 *time_step* — временной момент;
- 3 *tested_devices* и *total_devices* — параметры конфигурации сети;
- 4 *probability* — вероятность обнаружения одной атакованной ноды.

Если объект *Probe* проходит валидацию, рассчитываются:

1. *attack_prob* — вероятность атаки в текущий момент;
2. *attack_level* — классификация уровня угрозы.

На основе этих данных формируется объект *Result*, инкапсулирующий информацию для визуализации или аналитики.

4.3.3. Контроллер *AttacksController*

Контроллер *AttacksController* реализует интерфейс взаимодействия пользователя с моделью атаки (*Attack*) в рамках веб-приложения. Он отвечает за обработку HTTP-запросов, управление данными модели и вызов вспомогательных сервисов для выполнения расчётов. Контроллер структурирован по принципу REST и дополнен пользовательскими действиями для проведения аналитических расчётов.

Контроллер поддерживает стандартный набор операций над объектами класса *Attack*:

- *index* — загружает и отображает список всех атак. Поддерживает фильтрацию по типу атаки через параметр поиска *search*, а также постраничную навигацию (пагинацию);
- *new / create* — формирует и сохраняет новую запись об атаке. В случае ошибок — выводит сообщения и повторно отображает форму;
- *show* — отображает подробную информацию о конкретной атаке;
- *edit / update* — позволяет редактировать параметры существующей атаки. Обработка ошибок аналогична действию *create*;
- *destroy* — удаляет выбранную атаку из базы данных.

Дополнительное действие *calculate* позволяет пользователю рассчитать вероятность обнаружения атаки в заданный момент времени при конкретных параметрах:

1. Создаётся объект *Probe*, ассоциированный с выбранной атакой и параметрами, введенными пользователем: момент времени t , размер сети n , количество тестируемых устройств k и вероятность обнаружения одной атакованной ноды p .
2. Выполняется валидация параметров и, если она проходит успешно, рассчитываются:
 - вероятность обнаружения при одиночном тестировании;
 - вероятность при координированной стратегии;
 - фактическая вероятность атаки по модели.
3. Если параметры некорректны, пользователю выводится сообщение об ошибке, и происходит возврат на предыдущую страницу.

Действие *classification_table* инициирует массовый расчёт сценариев атак для набора параметров. Оно выполняет:

- 1 Чтение параметров симуляции из запроса (или использование значений по умолчанию).
- 2 Вызов сервисного объекта *AttackSimulationService*, который генерирует множество экземпляров *Probe* для всех комбинаций параметров (t, k, n) и заданных порогов вероятности (*probability_gt, probability_lt*).
- 3 Классификация результатов по уровням риска и передача их в представление.

Это действие даёт возможность провести аналитический обзор сценариев атак и оценить эффективность тестовых стратегий при различных условиях.

5. СТРУКТУРА ИНТЕРФЕЙСА

В этой главе представлены основные элементы интерфейса и поясняется их функциональное назначение.

Интерфейс построен на основе набора представлений, расположенных в каталоге `app/views/attacks`, каждое из которых отвечает за отдельный аспект взаимодействия пользователя с системой: от создания и редактирования параметров атак до визуализации результатов моделирования и запуска вычислений.

Представление `attacks/index` (рисунок 5.1) отвечает за отображение списка доступных моделей атак и управление ими. Страница предоставляет пользователю удобный интерфейс для просмотра, фильтрации, создания, редактирования и удаления атак.

В верхней части расположена форма поиска по типу атаки — она позволяет быстро отфильтровать записи по ключевым словам. Ниже отображается набор карточек: первая из них предназначена для добавления новой атаки, а остальные содержат информацию о существующих моделях. Каждая такая карточка показывает тип атаки и её параметры, а также предоставляет действия: редактирование, удаление и запуск моделирования через модальное окно. Внизу размещена пагинация для навигации по страницам.

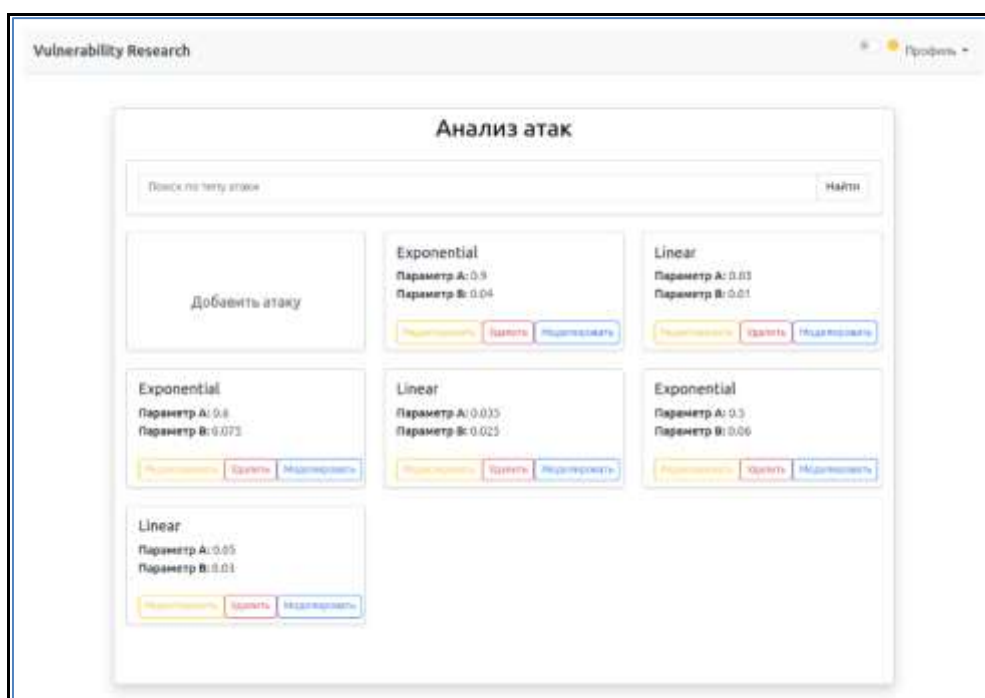


Рисунок 5.1 - Страница `attacks/index`

Представление calculationModal (рисунок 5.2) представляет собой модальное окно для ввода параметров расчета модели атаки. Пользователь получает удобный интерфейс с набором полей для задания основных параметров: времени моделирования, общего числа устройств, количества тестируемых устройств и вероятности обнаружения.

В верхней части модального окна размещен заголовок и кнопка закрытия. Ниже расположена форма с числовыми полями, где можно задать значения вручную. Дополнительно предусмотрен переключатель «Варьировать по времени», который при активации заменяет одно поле вероятности на два поля для задания вероятностей при разных условиях, что позволяет более гибко настроить модель. Внизу находятся кнопки «Рассчитать» для запуска вычислений и «Отмена» для закрытия окна без изменений. Поведение формы поддерживается скриптом: в зависимости от состояния переключателя данные либо отправляются на сервер, либо происходит перенаправление на таблицу классификации с параметрами.

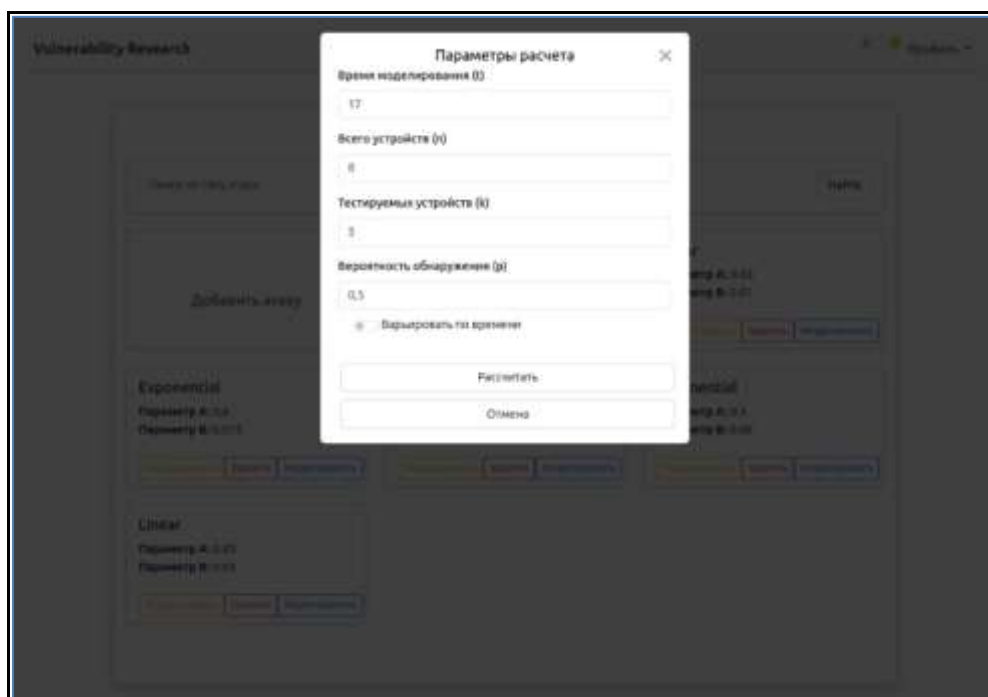


Рисунок 5.2 – Модальное окно для ввода параметров расчета

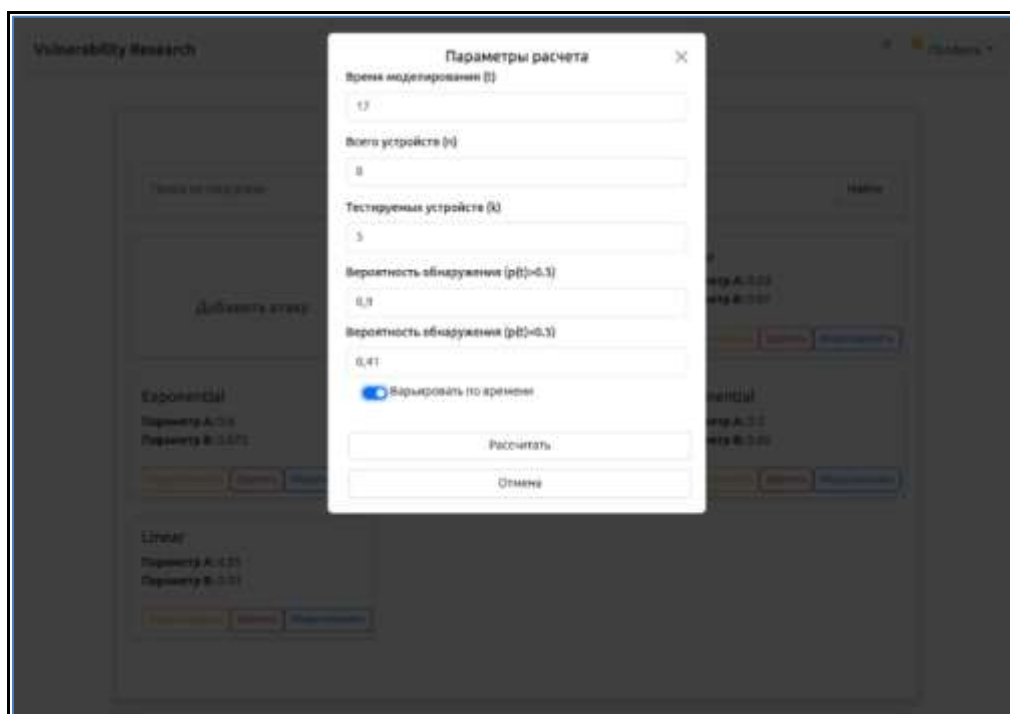


Рисунок 5.3 – Модальное окно для ввода параметров расчета при варьировании параметров

Представления edit и new используют общий паршал формы (рисунки 5.4, 5.5), который отвечает за создание и редактирование моделей атак. Интерфейс организован в виде карточки с заголовком, который динамически меняется в зависимости от действия: «Создать новую атаку» или «Редактировать атаку».

Форма содержит поля для выбора типа атаки из выпадающего списка и ввода числовых параметров A и B с валидацией на стороне клиента. При наличии ошибок валидации пользователю отображаются подсказки с пояснениями, которые обновляются динамически при изменении значений полей. Внизу размещены кнопки для сохранения изменений и отмены с возвратом к списку атак. Такой подход обеспечивает удобство и наглядность при работе с параметрами моделей.

The screenshot shows a web application window titled 'Vulnerability Research'. In the top right corner, there is a user profile icon and the text 'Профиль'. The main content area displays a form titled 'Редактировать атаку' (Edit Attack). The form contains three input fields: 'Тип атаки' (Attack Type) with the value 'Exponential', 'Параметр A' (Parameter A) with the value '0.9', and 'Параметр B' (Parameter B) with the value '2.04'. At the bottom of the form are two buttons: 'Обновить' (Update) and 'Отмена' (Cancel).

Рисунок 5.4 – Форма редактирования существующей атаки

The screenshot shows the same 'Vulnerability Research' application window. The main content area displays a form titled 'Создать новую атаку' (Create New Attack). The form contains three empty input fields: 'Тип атаки' (Attack Type), 'Параметр A' (Parameter A), and 'Параметр B' (Parameter B). At the bottom of the form are two buttons: 'Сохранить' (Save) and 'Отмена' (Cancel).

Рисунок 5.5 – Форма создания новой атаки

Представление `classification_table.html.erb` (рисунок 5.6) отображает результаты моделирования вероятности атаки и вероятности её обнаружения в виде комплексной таблицы и графиков. Пользователь получает удобный интерфейс для анализа динамики угроз и их детектирования на различных временных шагах.

В верхней части представления расположен заголовок с названием "Результаты моделирования". Основная часть содержит таблицу с

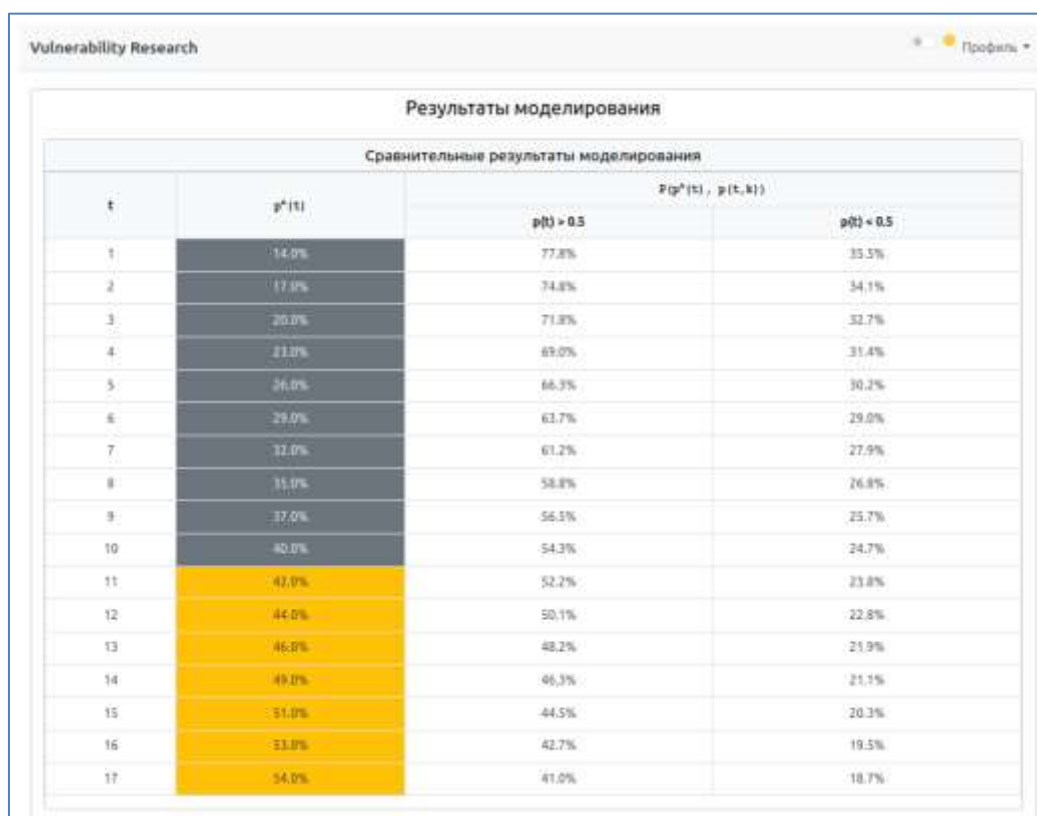
подробными данными по каждому временному шагу: среднее значение вероятности атаки и вероятности обнаружения для двух сценариев — когда вероятность обнаружения больше 0.5 и меньше 0.5. Значения вероятности отображаются в процентах с точностью до одного знака после запятой, при этом уровень угрозы визуально выделен цветом для удобства восприятия.

Ниже таблицы размещены два графика, реализованные с помощью библиотеки Chart.js. Первый график показывает динамику вероятности атаки и обнаружения для сценария с вероятностью обнаружения выше 0.5, второй — для сценария с вероятностью ниже 0.5. Ось X на графиках соответствует временным шагам, ось Y — процентной вероятности.

В нижней части страницы представлен блок с исходными параметрами моделирования: тип атаки, параметры модели, количество тестируемых и общих устройств, время моделирования и исходные вероятности обнаружения.

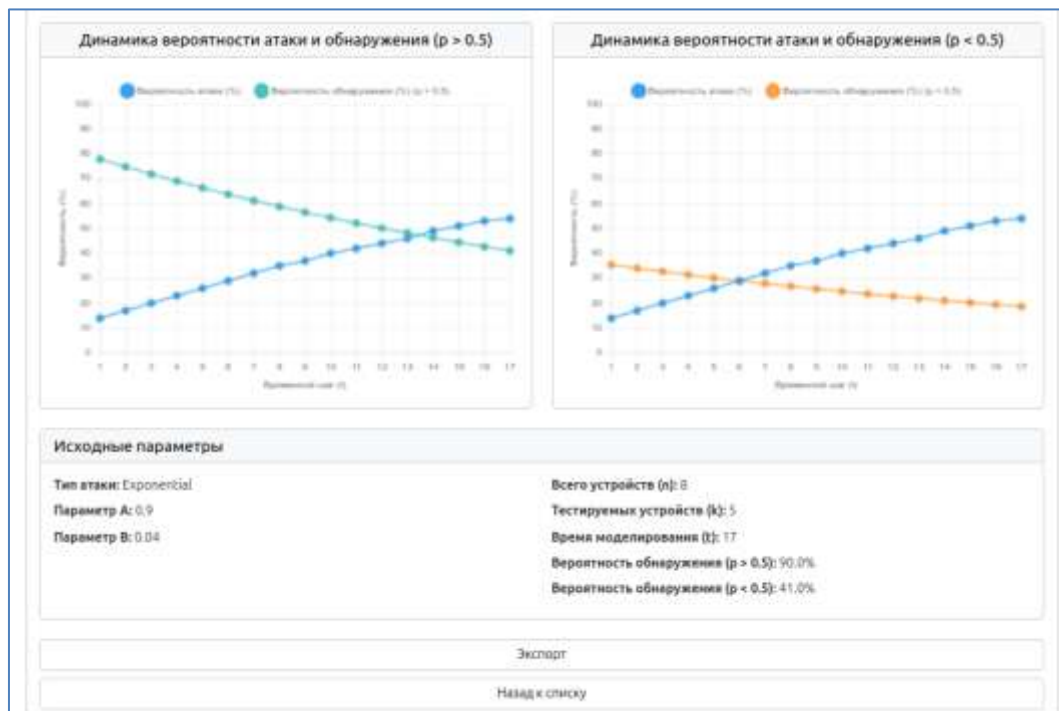
Кнопки "Экспорт" и "Назад к списку" расположены внизу страницы для удобного управления результатами.

Динамическое поведение графиков и отображение данных обеспечиваются встроенным JavaScript-кодом, который формирует и обновляет диаграммы на основе данных, переданных из контроллера.



Результаты моделирования			
Сравнительные результаты моделирования			
t	$p^k(t)$	$F(p^k(t), p(t, k))$	
		$p(t) > 0.5$	$p(t) < 0.5$
1	14.0%	77.8%	35.5%
2	17.0%	74.8%	34.1%
3	20.0%	71.8%	32.7%
4	21.0%	69.0%	31.4%
5	24.0%	66.3%	30.2%
6	29.0%	63.7%	29.0%
7	32.0%	61.2%	27.9%
8	35.0%	58.8%	26.8%
9	37.0%	56.5%	25.7%
10	40.0%	54.3%	24.7%
11	47.0%	52.2%	23.8%
12	44.0%	50.1%	22.8%
13	46.0%	48.2%	21.9%
14	49.0%	46.3%	21.1%
15	51.0%	44.5%	20.3%
16	53.0%	42.7%	19.5%
17	54.0%	41.0%	18.7%

Рисунок 5.6 – Страница результатов моделирования



Продолжение рисунка 5.6

6. СЦЕНАРИИ ПРОВЕДЕНИЯ ЭКСПЕРИМЕНТОВ

Для проведения модельных экспериментов в рамках поставленной задачи были разработаны и реализованы сценарии, учитывающие влияние параметров атак различной интенсивности, типов атак, вероятностей их появления, а также стратегий тестирования.

Для формирования реалистичных условий эксперимента были заданы параметры шести атак, различающихся по типу (линейная и экспоненциальная) и интенсивности (низкая, средняя, высокая), согласно индивидуальному варианту из таблицы 1.1 постановки задачи.

Таблица 6.1 – Исходные данные

Интенсивность	Тип атаки	a	b	Обозначение
Низкая (N=5)	Линейная	0.05	0.03	Лин_Н
	Экспоненциальная	0.5	0.06	Эксп_Н
Средняя (J=8)	Линейная	0.035	0.025	Лин_С
	Экспоненциальная	0.6	0.075	Эксп_С
Высокая (K=1)	Линейная	0.03	0.01	Лин_В
	Экспоненциальная	0.9	0.04	Эксп_В

Каждый тип атаки моделируется с использованием одной из следующих вероятностных функций отказа:

1. Линейная модель: $p^A(t) = at + b$;
2. Экспоненциальная модель: $p^A(t) = 1 - a \times \exp(-bt)$.

Вероятность появления каждого типа атаки фиксирована: $p(ai) = 0.33$.

Коллаборационное тестирование предполагает одновременное участие нескольких тестирующих устройств. Вероятность обнаружения уязвимости в этом случае обозначается как $P(p(t, k))$, где:

- t — временной шаг;
- $p(t)$ — вероятность однократного обнаружения уязвимости;
- k — мощность коллаборации (число устройств, участвующих в тестировании);

- n — общее количество доступных устройств.

В соответствии с индивидуальным вариантом (таблица 1.2), моделируются два случая:

- Высокая вероятность обнаружения: $p(t) = 0.9p(t)$;
- Низкая вероятность обнаружения: $p(t) = 0.41p(t)$;

Для каждого из этих случаев исследуется влияние параметров $k \in [1; 5]$, $n \in [1; 8]$ на вероятность группового обнаружения уязвимости. Вероятность $P(p(t, k))$ рассчитывается с учётом согласованных действий между тестирующими узлами.

Эксперименты проводились в следующей последовательности:

1. Для каждого типа атаки формируются значения функции отказа $p(t)$ в диапазоне $t \in [1; 17]$.
2. Для каждого временного шага t вычисляется вероятность обнаружения уязвимости при коллаборационном тестировании:

$$P(p^A(t), p(t, k)) = (1 - p^A(t)) \times (C_n^k \times p^k(t) \times (1 - p(t))^{n-k}.$$
3. Рассматриваются два режима:
 - при $p(t) = 0.9p(t)$ — высокий уровень обнаружения;
 - при $p(t) = 0.41p(t)$ — низкий уровень обнаружения.

7. РЕЗУЛЬТАТЫ ИССЛЕДОВАНИЯ

Эксперимент 1. Линейная атака, низкая интенсивность.

В первом сценарии моделируется поведение системы в условиях атаки линейного типа при минимальной интенсивности воздействия. Значения параметров атаки установлены как $a = 0,05$, $b = 0,03$. Моделирование проводилось на выборке из $n = 8$ устройств и $k = 5$ тестирующих. Моделирование проводится в течение 17 временных шагов.

Результаты эксперимента приведены на рисунке 7.1. В таблице отображена вероятность реализации атаки $p^A(t)$, а также итоговая вероятность её обнаружения при двух уровнях индивидуальной точности тестирования: высокой ($p(t) = 0,90$) и низкой ($p(t) = 0,41$).

Сравнительные результаты моделирования			
t	$p^A(t)$	$P(p^A(t), p(t, k))$	
		$p(t) > 0.5$	$p(t) < 0.5$
1	8.0%	82.0%	37.7%
2	13.0%	78.3%	35.7%
3	18.0%	73.0%	33.6%
4	23.0%	69.3%	31.6%
5	28.0%	64.0%	29.5%
6	33.0%	60.3%	27.5%
7	38.0%	55.0%	25.4%
8	43.0%	51.3%	23.4%
9	48.0%	46.0%	21.3%
10	53.0%	42.3%	19.3%
11	58.0%	37.0%	17.2%
12	63.0%	33.3%	15.2%
13	68.0%	28.0%	13.1%
14	73.0%	24.3%	11.1%
15	78.0%	19.0%	9.0%
16	83.0%	15.3%	7.0%
17	88.0%	10.0%	4.9%

Рисунок 7.1 – Моделирование линейной атаки: $a = 0.05$, $b = 0.03$

На рисунке 7.2 представлены графики изменения вероятности атаки и вероятности обнаружения уязвимостей во времени.

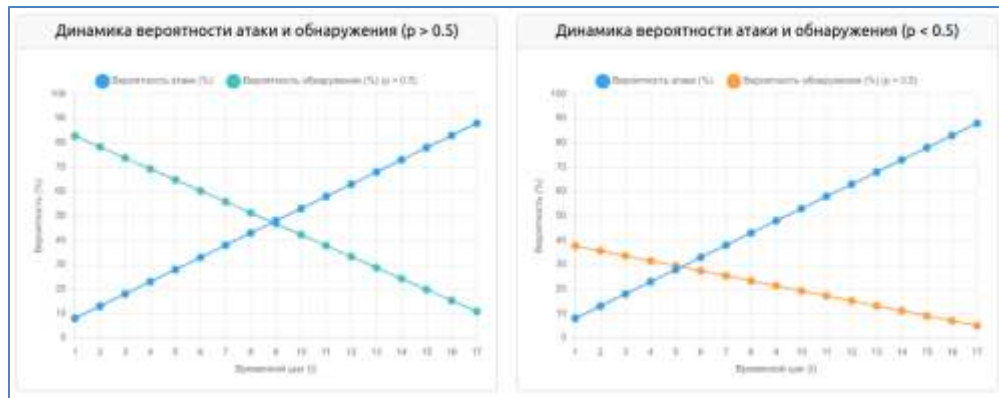


Рисунок 7.2 – Динамика изменения вероятности атаки и обнаружения

Эксперимент 2. Экспоненциальная атака, низкая интенсивность

Во втором эксперименте рассматривается экспоненциальный тип атаки с тем же числом устройств и тестирующих, но с параметрами $a = 0,5$, $b = 0,06$, задающими плавный рост активности угрозы.

На рисунке 7.3 представлены результаты моделирования. Для каждого временного шага рассчитана вероятность атаки $p^A(t)$, а также интегральная вероятность успешного обнаружения уязвимости при высокой и низкой точности индивидуального теста. Уже с первого шага видно, что значение $p^A(t)$ превышает 50%, однако эффективность обнаружения при низкой точности быстро снижается на фоне растущей угрозы.

Сравнительные результаты моделирования			
t	$p^A(t)$	$P(p^A(t), p(t, k))$	
		$p(t) > 0.5$	$p(t) < 0.5$
1	53.0%	42.4%	19.3%
2	56.0%	39.9%	18.2%
3	58.0%	37.6%	17.1%
4	61.0%	35.4%	16.1%
5	63.0%	33.3%	15.2%
6	65.0%	31.4%	14.3%
7	67.0%	29.6%	13.3%
8	69.0%	27.8%	12.7%
9	71.0%	26.2%	11.9%
10	73.0%	24.7%	11.3%
11	74.0%	23.3%	10.6%
12	76.0%	21.9%	10.0%
13	77.0%	20.6%	9.4%
14	78.0%	19.4%	8.9%
15	80.0%	18.3%	8.3%
16	81.0%	17.2%	7.8%
17	82.0%	16.2%	7.4%

Рисунок 7.3 – Моделирование экспоненциальной атаки: $a = 0.5$, $b = 0.06$

На рисунке 7.4 показаны соответствующие графики.

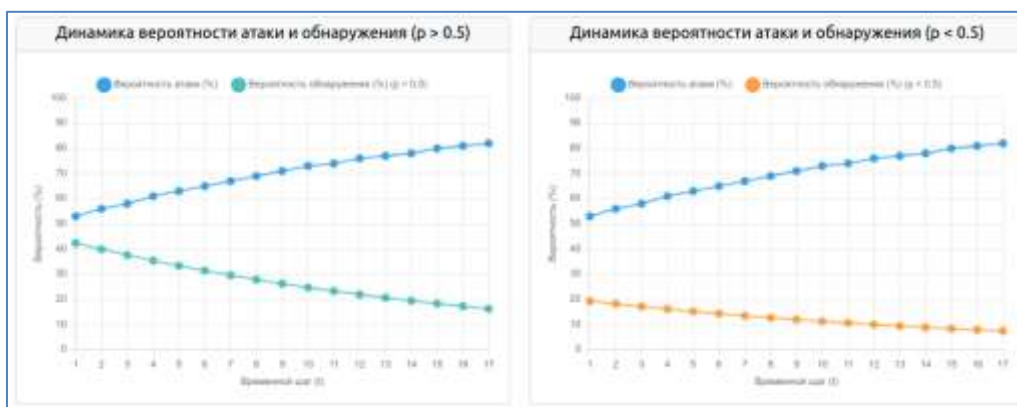


Рисунок 7.4 – Динамика изменения вероятности атаки и обнаружения

Эксперимент 3. Линейная атака, средняя интенсивность

В третьем сценарии исследуется поведение системы при линейной атаке со средним уровнем интенсивности, задаваемым параметрами $a = 0,035$, $b = 0,025$. Остальные условия моделирования остаются неизменными.

Результаты моделирования представлены на рисунке 7.5. Вероятность реализации атаки возрастает от 6% до 62%, что указывает на устойчивый рост угрозы во времени. При пониженной точности тестирования наблюдается заметное снижение вероятности успешного обнаружения уже с середины интервала.

t	$p^*(t)$	$P(p^*(t) - p(t, k))$	
		$p(t) > 0.5$	$p(t) < 0.5$
1	6.0%	84.6%	38.3%
2	10.0%	81.5%	37.1%
3	13.0%	78.3%	35.7%
4	17.0%	75.2%	34.2%
5	20.0%	72.0%	32.8%
6	24.0%	68.9%	31.4%
7	27.0%	65.7%	29.9%
8	31.0%	62.6%	28.5%
9	34.0%	59.4%	27.1%
10	38.0%	56.3%	25.6%
11	41.0%	53.1%	24.2%
12	45.0%	50.0%	22.8%
13	48.0%	46.8%	21.3%
14	52.0%	43.7%	19.9%
15	55.0%	40.5%	18.5%
16	59.0%	37.4%	17.0%
17	62.0%	34.2%	15.6%

Рисунок 7.5 – Моделирование линейной атаки: $a = 0.035$, $b = 0.025$

Графики на рисунке 7.6 визуализируют, как развиваются атака и вероятности обнаружения уязвимостей с течением времени. Особенно наглядно прослеживается влияние точности тестов на способность своевременно выявлять угрозу по мере роста её интенсивности.

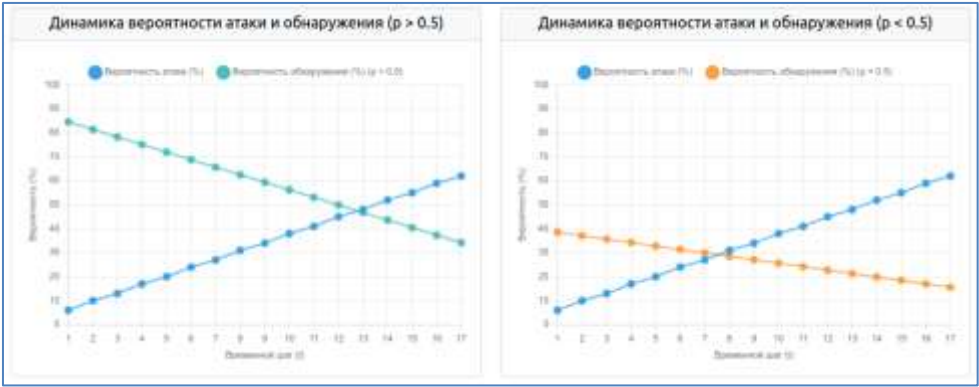


Рисунок 7.6 – Динамика изменения вероятности атаки и обнаружения

Эксперимент 4. Экспоненциальная атака, средняя интенсивность

Четвёртый эксперимент моделирует экспоненциальную атаку с параметрами средней интенсивности: $a = 0,6$, $b = 0,075$.

Результаты численного моделирования сведены в таблице на рисунке 7.7. По мере развития атаки наблюдается быстрое насыщение — вероятность её реализации растёт с 44% до 83%.

Сравнительные результаты моделирования			
t	$p^*(t)$	$P(p^*(t) : p(t, k))$	
		$p(t) > 0.5$	$p(t) < 0.5$
1	44.0%	50.1%	22.8%
2	48.0%	46.5%	21.2%
3	52.0%	43.1%	19.0%
4	56.0%	40.0%	18.2%
5	59.0%	37.1%	16.9%
6	62.0%	34.4%	15.7%
7	65.0%	31.9%	14.0%
8	67.0%	29.0%	13.3%
9	69.0%	27.5%	12.5%
10	72.0%	25.5%	11.0%
11	74.0%	23.7%	10.0%
12	76.0%	22.0%	10.0%
13	77.0%	20.4%	9.3%
14	79.0%	18.9%	8.0%
15	81.0%	17.5%	8.0%
16	82.0%	16.3%	7.4%
17	83.0%	15.1%	6.9%

Рисунок 7.7 – Моделирование экспоненциальной атаки: $a = 0,6$, $b = 0,075$

На рисунке 7.8 отображена динамика изменений вероятностей атаки и успешного обнаружения уязвимостей во времени. Визуализация подчёркивает, как рост интенсивности угрозы постепенно снижает эффективность коллаборационного тестирования, особенно в условиях ограниченной точности.

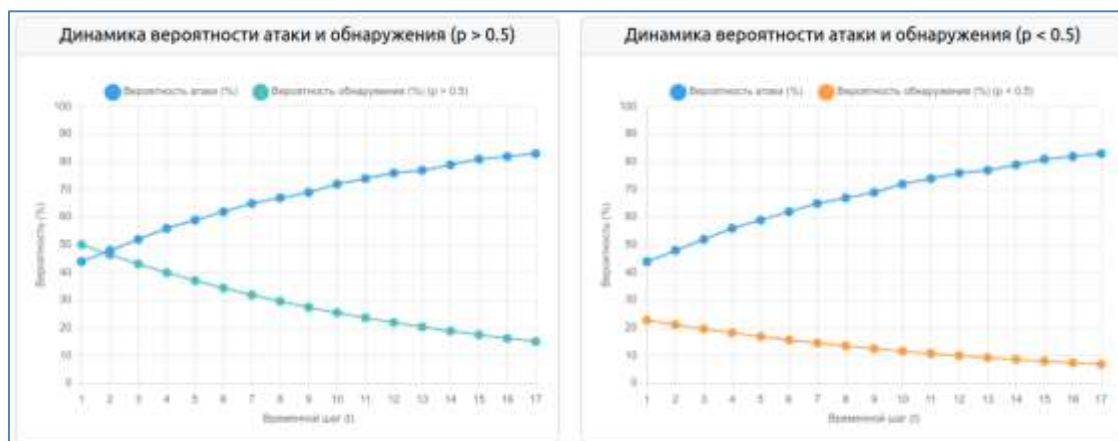


Рисунок 7.8 – Динамика изменения вероятности атаки и обнаружения

Эксперимент 5. Линейная атака, высокая интенсивность

В пятом эксперименте рассматривается сценарий линейной атаки с интенсивно нарастающей угрозой. Значения параметров установлены как $a = 0,03$, $b = 0,01$. Прочие условия моделирования остаются неизменными.

Результаты моделирования представлены в таблице на рисунке 7.9. По мере увеличения времени наблюдается стабильный рост вероятности атаки $p^A(t)$, начиная с 4% на первом шаге и достигая 52% к завершению периода. Сравнительный анализ эффективности коллаборационного тестирования при двух уровнях точности показывает, что преимущество первого варианта сохраняется на всём протяжении, но с каждым шагом обе кривые уверенно снижаются.

Сравнительные результаты моделирования			
t	$p^*(t)$	$P(p^*(t), p(t, k))$	
		$p(t) > 0.5$	$p(t) < 0.5$
1	4.0%	86.4%	39.4%
2	7.0%	83.7%	38.1%
3	10.0%	81.0%	36.9%
4	13.0%	78.3%	35.7%
5	16.0%	75.6%	34.4%
6	19.0%	72.9%	33.2%
7	22.0%	70.2%	32.0%
8	25.0%	67.5%	30.8%
9	28.0%	64.8%	29.5%
10	31.0%	62.1%	28.3%
11	34.0%	59.4%	27.1%
12	37.0%	56.7%	25.8%
13	40.0%	54.0%	24.6%
14	43.0%	51.3%	23.4%
15	46.0%	48.6%	22.1%
16	49.0%	45.9%	20.9%
17	52.0%	43.2%	19.7%

Рисунок 7.9 – Моделирование линейной атаки: $a = 0,03$, $b = 0,01$

На рисунке 7.10 представлена визуализация полученных данных. Видно, как равномерный рост вероятности атаки приводит к постепенному снижению шансов своевременного обнаружения уязвимостей, особенно при сниженной точности индивидуального теста. Уже к середине моделируемого периода эффективность тестирования при $p(t) = 0,41$ опускается ниже 30%.

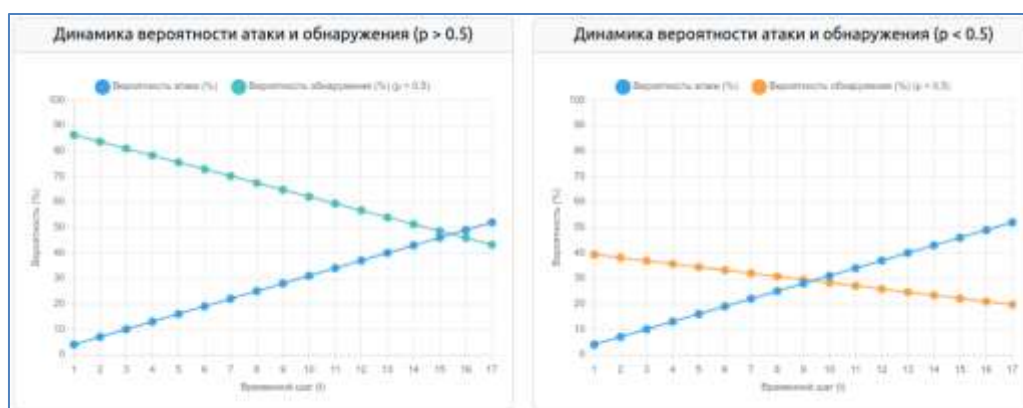


Рисунок 7.10 — Динамика изменения вероятности атаки и обнаружения

Эксперимент 6. Экспоненциальная атака, высокая интенсивность

Шестой эксперимент иллюстрирует поведение системы при воздействии экспоненциальной атаки с высокой динамикой роста. Параметры модели заданы как $a = 0,9$, $b = 0,04$.

Результаты численного моделирования представлены в таблице на рисунке 7.11.

k	$p^k(\%)$	$P(p^k(t), p(t, k))$	
		$p(t) > 0.5$	$p(t) < 0.5$
1	14.0%	77.3%	35.5%
2	17.0%	74.8%	34.1%
3	20.0%	71.5%	32.7%
4	23.0%	69.0%	31.4%
5	26.0%	66.3%	30.2%
6	28.0%	63.7%	29.0%
7	32.0%	61.2%	27.9%
8	35.0%	58.8%	26.8%
9	37.0%	56.5%	25.7%
10	40.0%	54.3%	24.7%
11	42.0%	52.2%	23.8%
12	44.0%	50.1%	22.8%
13	46.0%	48.2%	21.9%
14	48.0%	46.3%	21.1%
15	51.0%	44.3%	20.3%
16	53.0%	42.7%	19.5%
17	54.0%	41.0%	18.7%

Рисунок 7.11 – Моделирование экспоненциальной атаки: $a = 0,09$, $b = 0,04$

Динамика изменения вероятностей визуализирована на рисунке 7.12. Чётко прослеживается экспоненциальный рост угрозы, в то время как вероятность обнаружения уязвимости снижается независимо от выбранного уровня точности. Особенно заметно это при низком значении $p(t)$, где уже к середине интервала значение суммарной вероятности обнаружения опускается ниже 25%.

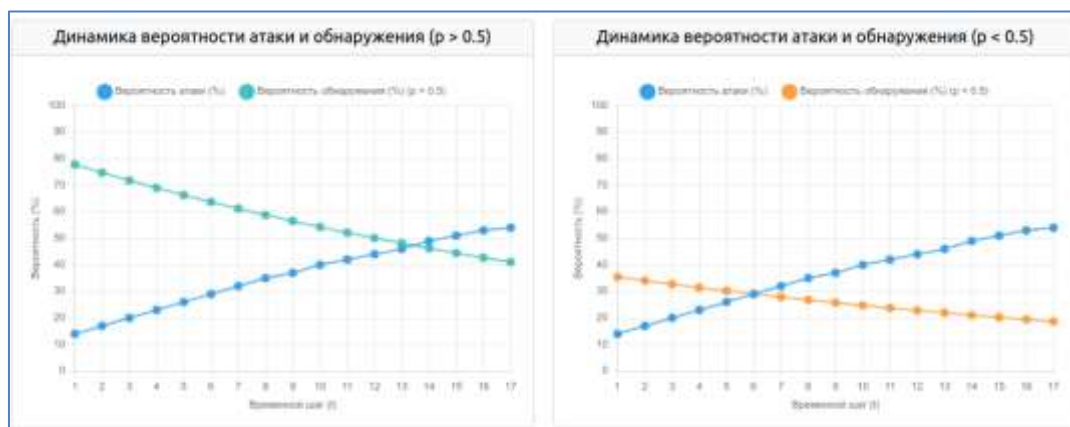


Рисунок 7.12 — Динамика вероятности атаки и обнаружения

Для обобщения результатов всех проведённых сценариев в таблице 7.1 приведена сводная классификация, отражающая зависимость вероятности атаки от времени и интенсивности. Представлены как линейные, так и экспоненциальные модели при различных уровнях агрессии. Эта таблица позволяет сравнить темпы нарастания угрозы и оценить, насколько различная динамика атак влияет на устойчивость системы к отказам.

Таблица 7.1 – Сводная классификация вероятности атак при различных типах и интенсивностях угроз на протяжении временного интервала моделирования.

Функция	$p^A(t) = at + b$			$p^A(t) = 1 - a \times \exp(-bt)$		
t \ интенсивность атаки	Лин Н	Лин С	Лин В	Ехр Н	Ехр С	Ехр В
1	0,08	0,06	0,04	0,53	0,44	0,14
2	0,13	0,10	0,07	0,56	0,48	0,17
3	0,18	0,13	0,10	0,58	0,52	0,20
4	0,23	0,17	0,13	0,61	0,56	0,23
5	0,28	0,20	0,16	0,63	0,59	0,26
6	0,33	0,24	0,19	0,65	0,62	0,29
7	0,38	0,27	0,22	0,67	0,65	0,32
8	0,43	0,31	0,25	0,69	0,67	0,35
9	0,48	0,34	0,28	0,71	0,69	0,37
10	0,53	0,38	0,31	0,73	0,72	0,40
11	0,58	0,41	0,34	0,74	0,74	0,42
12	0,63	0,45	0,37	0,76	0,76	0,44
13	0,68	0,48	0,40	0,77	0,77	0,46
14	0,73	0,52	0,43	0,78	0,79	0,49
15	0,78	0,55	0,46	0,80	0,81	0,51
16	0,83	0,59	0,49	0,81	0,82	0,53
17	0,88	0,62	0,52	0,82	0,83	0,54

При сравнении двух типов атак — линейных и экспоненциальных — видно, что экспоненциальные угрозы обладают более агрессивной начальной фазой роста. Это делает их особенно критичными в системах, чувствительных к раннему обнаружению. Линейные сценарии, напротив, позволяют выиграть время за счёт равномерного и предсказуемого нарастания риска.

На графике ниже отображена динамика вероятности реализации атаки $p_A(t)$ во времени для различных типов атак (линейных и экспоненциальных) и уровней их интенсивности (низкой, средней, высокой).

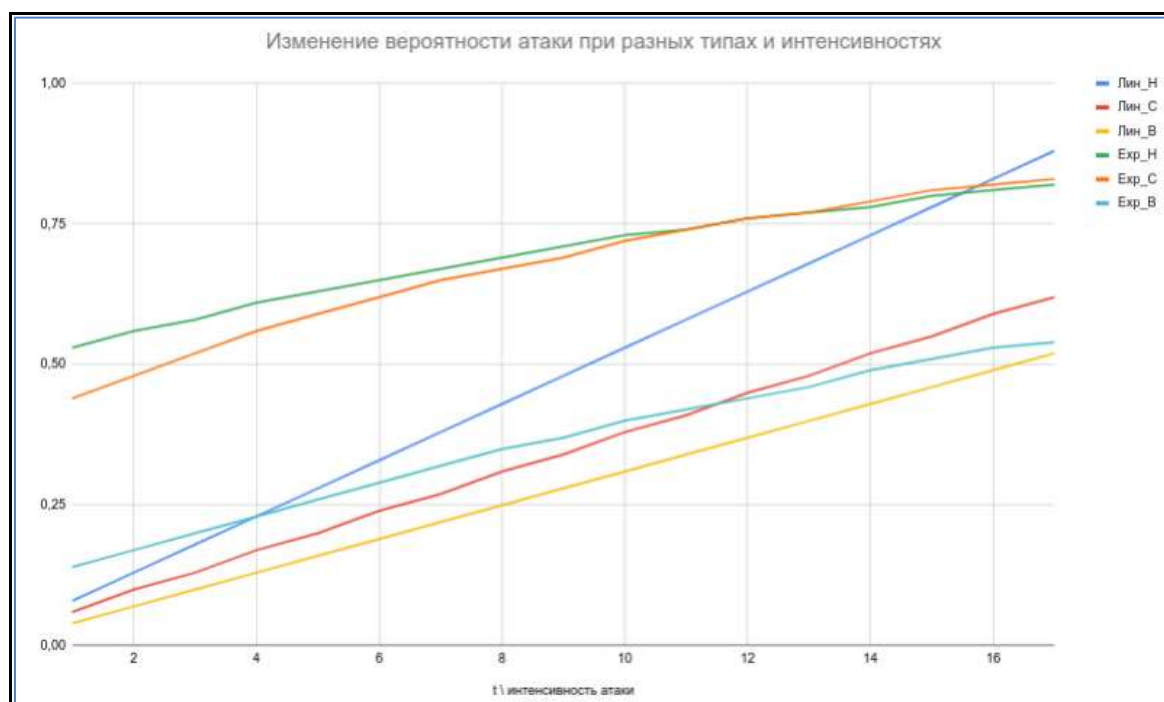


Рисунок 7.12 – Динамика вероятности атаки во времени при различных типах, уровнях и интенсивностях

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана программная система, реализующая моделирование атак на информационно-измерительные сети и оценку эффективности их обнаружения. Система позволяет исследовать влияние различных параметров атак (типа, интенсивности, вероятности появления) и стратегий тестирования (одиночного и коллаборационного) на вероятность обнаружения уязвимостей.

Проведенные эксперименты продемонстрировали зависимость эффективности обнаружения от типа атаки, её интенсивности и точности индивидуального тестирования. Экспоненциальные атаки показали более агрессивную динамику роста вероятности по сравнению с линейными, что требует более оперативного реагирования систем безопасности. Коллаборационное тестирование продемонстрировало преимущество перед одиночным, особенно при высокой точности обнаружения. Однако, по мере роста интенсивности атаки, эффективность обоих методов снижается, что подчеркивает необходимость в адаптивных стратегиях тестирования.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Барсегян, А. А. Анализ данных и процессов: учеб. пособие / А. А. Барсегян, М. С. Куприянов, И. И. Холод, М. Д. Тесс, С. И. Елизаров. — 3-е изд., перераб. и доп. — СПб.: БХВ-Петербург, 2009. — 512 с.: ил. + CD-ROM — (Учебная литература для вузов)
2. Боев В. Д. Компьютерное моделирование: Пособие для практических занятий, курсового и дипломного проектирования в AnyLogic7: — СПб.: ВАС, 2014. — 432 с.
3. Вуколов, Э. А. Основы статистического анализа. Практикум по статистическим методам и исследованию операций с использованием пакетов STATISTICA и EXCEL: Учебное пособие. — М.: ФОРУМ: ИНФРА-М, 2004. — 464 с. — ISBN 5-8199-0141-X, 5-16-002003-9. (см. стр. 8–18)
4. Кремер Н., Путко Б. Эконометрика: Учебник для вузов. — М.: ЮНИТИ-ДАНА, 2003. — 311 с. (см. стр. 9–24)
5. Рутковская Д., Пилиньский М., Рутковский Л. Нейронные сети, генетические алгоритмы и нечеткие системы / Пер. с польск. И. Д. Рудинского. — М.: Горячая линия — Телеком, 2008. — 452 с., ил.
6. Дасгупта Д. Искусственные иммунные системы и их применение / Пер. с англ. А. А. Романюха. — М.: Физматлит, 2006. — 344 с.
7. Практикум по курсу «Статистика» (в системе STATISTICA) / В. Н. Салин, Э. Ю. Чурилова. — М.: Издательский Дом «Социальные отношения», Изд-во «Перспектива», 2002. — 188 с. (см. стр. 71–96)
8. Айвазян С. А. Прикладная статистика. Основы эконометрики: Учебник для вузов: В 2 т. 2-е изд., испр. — Т. 2: Основы эконометрики. — М.: ЮНИТИ-ДАНА, 2001. — 432 с. (см. стр. 49–53)
9. Халафян А. А. STATISTICA 6. Статистический анализ данных. 3-е изд. Учебник. — М.: ООО «Бином-Пресс», 2007. — 512 с. — ISBN 978-5-9518-0215-6. (см. стр. 13–51, 59–81)

ПРИЛОЖЕНИЕ А

Код модели Attack:

```

class Attack < ApplicationRecord
  self.inheritance_column = nil

  enum attack_type: {
    linear: "linear",
    exponential: "exponential"
  }, _default: "linear"

  validates :parameter_a, :parameter_b,
    numericality: {
      greater_than_or_equal_to: 0,
      less_than_or_equal_to: 1,
      format: { with: /\A\d+(?:\.\d{1,3})?\z/ }
    }
  validates :attack_type, inclusion: { in: attack_types.keys }

  def linear_probability(time_step)
    (parameter_a * time_step + parameter_b).clamp(0.0, 1.0)
  end

  # Вероятность атаки на основе экспоненциальной функции
  (Формула 12 в МУ)
  def exponential_probability(time_step)
    (1 - parameter_a * Math.exp(-parameter_b *
time_step)).clamp(0.0, 1.0)
  end

  def attack_level(time_step)
    prob = probability_at(time_step) # Получаем вероятность в
данный момент времени
    if prob < 0.4
      { level: "Низкий", color: "gray" }
    elsif prob < 0.7
      { level: "Средний", color: "yellow" }
    else
      { level: "Высокий", color: "red" }
    end
  end

  def probability_at(time_step)
    attack_type == "linear" ? linear_probability(time_step) :
exponential_probability(time_step)
  end
end

```

Код модели Probe:

```

class Probe < ApplicationRecord
  belongs_to :attack

  # Валидации
  validates :time_step, :total_devices, :tested_devices,
    numericality: { only_integer: true, greater_than: 0 }
  validates :probability, numericality: { in: 0.0..1.0 }

  # Вероятность обнаружения при однократном тестировании с
  учетом противодействия (формула 4 в МУ)
  def single_test_probability
    (1 - attack_probability) * probability * (1 -
probability)**(tested_devices - 1)
  end

  # Вероятность обнаружения при коллаборационной стратегии с
  учетом противодействия (Формула 5 в МУ)
  def collaborative_test_probability
    (1 - attack_probability) * binomial_coefficient *
(probability**tested_devices) * ((1 -
probability)**(total_devices - tested_devices))
  end

  private

  def attack_probability
    attack.probability_at(time_step) # Вызываем метод
probability_at у связанной модели Attack
  end

  # Метод для вычисления биномиального коэффициента
  def binomial_coefficient
    return 0 if tested_devices > total_devices # Если
количество тестируемых больше общего, возвращаем 0
    (1..total_devices).to_a.combination(tested_devices).size #
Вычисляем количество сочетаний
  end
end

```

Код сервисного класса AttackSimulationService:

```

class AttackSimulationService
  def self.run(attack, time_steps, total_devices,
    tested_devices, probability_gt, probability_lt)
    return { probability_gt: [], probability_lt: [] } unless
    attack.present?
    {
      probability_gt: simulate(attack, time_steps,
        total_devices, tested_devices, probability_gt),
      probability_lt: simulate(attack, time_steps,
        total_devices, tested_devices, probability_lt)
    }
  end
  def self.simulate(attack, time_steps, total_devices,
    tested_devices, probability)
    results = []
    time_steps_range = (1..(time_steps || 17)).to_a
    total_devices_range = (1..(total_devices || 8)).to_a
    tested_devices_range = (1..(tested_devices || 5)).to_a
    time_steps_range.each do |t|
      tested_devices_range.each do |tested|
        total_devices_range.each do |total|
          next if tested > total
          attack_prob = attack.probability_at(t)
          probe = Probe.new(
            attack: attack,
            time_step: t,
            tested_devices: tested,
            total_devices: total,
            probability: probability.to_f
          )
          next unless probe.valid?
          attack_level = attack.attack_level(t)
          results << Result.new(
            attack.attack_type,
            t,
            tested,
            total,
            attack_prob,
            attack_level[:level],
            attack_prob.round(2)
          )
        end
      end
    end
    results
  end
end

```

Код контроллера AttacksController:

```
class AttacksController < ApplicationController
  before_action :set_attack, only: [:show, :edit, :update,
:destroy, :calculate]
  before_action :validate_calculation_params, only: :calculate

  def index
    @attacks = Attack.order(created_at: :desc)
    @attacks = @attacks.where("attack_type LIKE ?",
"%#{params[:search]}%") if params[:search].present?
    @attacks = @attacks.page(params[:page]).per(10)
  end

  def new
    @attack = Attack.new
  end

  def create
    @attack = Attack.new(attack_params)
    if @attack.save
      redirect_to attacks_path, notice: "Атака успешно
создана"
    else
      flash.now[:alert] = "Ошибка при создании атаки:
#{@attack.errors.full_messages.join(', ')}"
      render :new, status: :unprocessable_entity
    end
  end

  def show
  end

  def edit
  end

  def update
    if @attack.update(attack_params)
      redirect_to attacks_path, notice: "Атака успешно
обновлена"
    else
      flash.now[:alert] = "Ошибка при обновлении атаки:
#{@attack.errors.full_messages.join(', ')}"
      render :edit, status: :unprocessable_entity
    end
  end

  def destroy
    @attack.destroy
    redirect_to attacks_url, notice: "Атака успешно удалена."
```

```

end

def calculate
  # Создаем временный объект Probe для расчетов
  @probe = build_probe_for_calculation

  @time_step = @probe.time_step
  @total_devices = @probe.total_devices
  @tested_devices = @probe.tested_devices
  @probability = @probe.probability

  if @probe.valid?
    perform_probability_calculations
    render :calculate
  else
    handle_calculation_error
  end
end

def classification_table
  @attack = Attack.find(params[:id])
  @time_steps = params[:time_steps].present? ?
params[:time_steps].to_i : 17
  @total_devices = params[:total_devices].present? ?
params[:total_devices].to_i : 8
  @tested_devices = params[:tested_devices].present? ?
params[:tested_devices].to_i : 5
  @probability = params[:probability].present? ?
params[:probability].to_f : 0.5
  @probability_gt = params[:probability_gt] ?
params[:probability_gt].to_f : 0.9
  @probability_lt = params[:probability_lt] ?
params[:probability_lt].to_f : 0.41
  @results = AttackSimulationService.run(
    @attack,
    @time_steps.to_i,
    @total_devices.to_i,
    @tested_devices.to_i,
    @probability_gt.to_f,
    @probability_lt.to_f
  ) || { probability_gt: [], probability_lt: [] } # Защита
от nil
  rescue ActiveRecord::RecordNotFound
    redirect_to attacks_path, alert: "Атака не найдена"
  end

private

def set_attack
  @attack = Attack.find_by(id: params[:id])
  redirect_with_alert("Атака не найдена") unless @attack
end

```

```

end

def validate_calculation_params
  required_params = [:time_step, :total_devices,
:tested_devices, :probability]

  required_params.each do |param|
    unless valid_parameter?(params[param])
      redirect_with_alert("Неверные параметры расчета") and
return
    end
  end
end

def valid_parameter?(value)
  value.present? && value.to_f >= 0
end

def attack_params
  params.require(:attack).permit(:attack_type, :parameter_a,
:parameter_b)
end

def build_probe_for_calculation
  Probe.new(
    attack: @attack,
    time_step: params[:time_step].to_i,
    total_devices: params[:total_devices].to_i,
    tested_devices: params[:tested_devices].to_i,
    probability: params[:probability].to_f
  )
end

def perform_probability_calculations
  @single_prob = @probe.single_test_probability
  @collab_prob = @probe.collaborative_test_probability
  @attack_probability =
@attack.probability_at(@probe.time_step)
end

def handle_calculation_error
  error_message = "Ошибка в параметрах расчета:
#{@probe.errors.full_messages.join(', ')}"
  redirect_with_alert(error_message)
end

def redirect_with_alert(message)
  flash[:alert] = message
  redirect_to attacks_path
end
end

```